Document number	P0742R0
Date	2017-07-14
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Wording for Networking PDTS ballot comment resolutions

Introduction

This paper presents proposed changes in response to some of the National Body comments for PDTS 19216 from N4643.

Changes

NB comment 007 (GB-2)

> "No constructor ... shall exit via an exception" over-specification. This is probably a copy/paste error introduced when this text was copied from the standard (I believe from [allocator.requirements]). The intent is only that copy and move constructors shall not throw, and I think that is already covered by "copy operation, move operation". Other constructors not covered by the ProtoAllocator requirements should be allowed to throw.
>
> **Proposed change:** Delete "constructor," so that the sentence reads "No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception."

The comment is correct, the text was copied from the requirements on allocator pointer types, and imposes restrictions on proto-allocators that are not present on C++ allocators (see LWG 2455). For consistency with the allocator requirements in the IS, non-copy/move constructors should both be allowed to throw.

NB comment 009 (GB-4)

> "No constructor ... shall exit via an exception" over-specification. This is probably a copy/paste error introduced when this text was copied from the standard (I believe from [allocator.requirements]). The intent is only that copy and move constructors shall not throw, and I think that is already covered by "copy operation, move operation". Other constructors not covered by the Executor requirements should be allowed to throw.
>
> **Proposed change:** Delete "constructor," so that the sentence reads "No comparison operator, copy operation, move operation, swap operation, or member functions context, on_work_started, and on_work_finished on these types shall exit via an exception."

Similar to 007.

NB comment 010 (GB-5)

> Executor requirements table refers to undefined name 'Func'. The Executor requirements table entries for 'dispatch', 'post' and 'defer' refer to an undefined name 'Func', in 'DECAY_COPY(forward(f))'. This name is not listed in the paragraph preceding the table.

> **Proposed change:** Fix the requirements table so that either the name 'Func' is defined, or so that the requirements for 'dispatch', 'post' and 'defer' are specified without referring to this type.

Define `Func` and rephrase slightly to avoid using "callable" which is a term of art.

I also noticed that the dispatch functions require an Allocator, but this is a perfect example of something that only needs a proto-allocator, because it will be rebound anyway. I fixed that as a drive-by.

NB comment 013 (US-12)

> "user-defined function objects" - user defined is possibly over-specification. In other parts of the TS it refers to types not specified by this TS. If there are function objects defined in the standard, they should also be destroyed.
>
> **Proposed change:** remove "user-defined"

Accept the change.

NB comment 020 (GB-11)

> Consider adding noexcept to buffer sequence requirements [buffer.reqmts.mutablebuffersequence], [buffer.reqmts.constbuffersequence], [buffer.seq.access]
>
> **Proposed change:** Adding "Shall not exit via an exception." to the the requirements for net::buffer_sequence_begin(x) and 'net::buffer_sequence_end(x).
> Requiring that the conversion of the iterator value type to const_buffer or mutable_buffer should not exit via exception.
> (And perhaps place the same requirement on the iterator traversal and dereference too, although I'm not sure if this is already implied elsewhere in the standard?)
> Adding noexcept to the buffer sequence access functions.
> The current implementation in asio assumes that these never throw, and in general I think low level buffer operations should not throw.

It seems reasonable to forbid these operations from throwing, and therefore good to add `noexcept` to the access functions (which have wide contracts).

Casey Carter also noticed that the **return type** column requires the value type to be convertible to the buffer, but it should be the reference type.

NB comment 024 (GB-13)

> The derived socket types basic_datagram_socket and basic_stream_socket should specify that their native_handle_type is the same as basic_socket::native_handle_type.
>
> **Proposed change:** Add such specification

This ensures that the derived class constructor can pass a native handle argument to the base class constructor.

Additionally, the PDTS is missing wording to say that when effects are described as if by POSIX `foobar(native_handle())` that it is assumed that `native_handle()` returns an `int`!

NB comment 028 (GB-17)

> [socket.acceptor.cons] move ctor missing postcondition. The postconditions for the basic_socket_acceptor move ctor don't have any postconditions on native_handle().

Proposed change: Add "native_handle() returns the prior value of rhs.native_handle()."

Accept the change.

NB comment 029 (GB-18)

[socket.streambuf.cons] Add missing error() postconditions. The basic_socketstreambuf constructors do not give any postconditions for the ec_ member.

Proposed change: Add "and !error()" to paragraphs 2 and 4. Add "error() == rhs_p.error()" and "!rhs_a.error()" to paragraph 6.

Accept the change.

NB comment 030 (GB-19)

[socket.streambuf.cons] Cover changes to rhs in operator= effects. The assignment operator for basic_socketstreambuf doesn't state the effects on the RHS.

Proposed change: Change "*this has the observable state it would have had if it had been move constructed from rhs" to "*this and rhs have the observable state they would have had if *this had been move constructed from rhs"

Accept the change.

NB comment 033 (GB-20)

Shorten ip::resolver_errc enumerator names. These enumerator names predate enum classes as a language feature and were so named to eliminate likely name clashes with other entities in the same namespace. The enumerator "host_not_found_try_again" is particularly long and could be shortened.

Proposed change: Rename the enumerator "host_not_found_try_again" to "try_again".

This was discussed by LEWG: "Needs a paper looking at the whole set of enum names systematically". This is that paper.

The PDTS contains the following enumeration types and enumerators:

```
enum class fork_event {
  prepare,
  parent,
  child
};

enum class stream_errc {
  eof = an implementation defined non-zero value ,
  not_found = an implementation defined non-zero value
};

enum class socket_errc {
  already_open = an implementation defined non-zero value ,
  not_found = an implementation defined non-zero value
};

enum class resolver_errc {
  host_not_found = an implementation-defined non-zero value , // EAI_NONAME
  host_not_found_try_again = an implementation-defined non-zero value , // EAI_AGAIN
  service_not_found = an implementation-defined non-zero value // EAI_SERVICE
};
```

Of these, only the enumerators for `resolver_errc` are lengthy. As there are two different "not found" errors they can't reasonably be shortened to simply `not_found`. They could conceivably be `no_name` and `bad_service`, but the current names seem just as good to me.

That leaves just `host_not_found_try_again`, which is what the NB comment objects to. POSIX defines the EAI_AGAIN error as "The name could not be resolved at this time. Future attempts may succeed." I think `try_again` is a good fit for that.

Proposed Wording

All changes are relative to N4656.

Modify 13.2.1 [async.reqmts.proto.allocator] paragraph 1 as shown:

> A type `A` meets the proto-allocator requirements if `A` is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where `U` is an object type. *[Note:* For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. *--end note]* No ~~constructor,~~ comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

Modify 13.2.2 [async.reqmts.executor] paragraph 3 as shown:

> No ~~constructor,~~ comparison operator, copy operation, move operation, swap operation, or member functions `context`, `on_work_started`, and `on_work_finished` on these types shall exit via an exception.

Modify 13.2.2 [async.reqmts.executor] paragraph 6 as shown:

> In Table 4, `x1` and `x2` denote (possibly const) values of type `X`, `mx1` denotes an xvalue of type `X`, `f` denotes a function object of `MoveConstructible` (C++Std [moveconstructible]) ~~function object~~ type `Func` such that `f()` is a valid expression ~~callable with zero arguments~~, `a` denotes a (possibly const) value of type `A` meeting the `ProtoAllocator` requirements (~~C++Std [allocator.requirements]~~ 12.2.1 [async.reqmts.proto.allocator]), and `u` denotes an identifier.

Modify 13.2.4 [async.reqmts.service] paragraph 5 as shown:

> A service's `shutdown` member function shall destroy all copies of ~~user-defined~~ function objects that are held by the service.

Modify 16.1 [buffer.synop] to add `noexcept` to the signatures:

> *// buffer sequence access:*

```
const mutable_buffer* buffer_sequence_begin(const mutable_buffer& b) noexcept;
const const_buffer* buffer_sequence_begin(const const_buffer& b) noexcept;
const mutable_buffer* buffer_sequence_end(const mutable_buffer& b) noexcept;
const const_buffer* buffer_sequence_end(const const_buffer& b) noexcept;
template<class C> auto buffer_sequence_begin(C& c) noexcept -> decltype(c.begin());
template<class C> auto buffer_sequence_begin(const C& c) noexcept -> decltype(c.begin());
template<class C> auto buffer_sequence_end(C& c) noexcept -> decltype(c.end());
template<class C> auto buffer_sequence_end(const C& c) noexcept -> decltype(c.end());
```

Modify 16.2.1 [buffer.reqmts.mutablebuffersequence] Table 12 by adding to the third column of the first row:

> **expression**

`net::buffer_sequence_begin(x)`
`net::buffer_sequence_end(x)`

return type

An iterator type meeting the requirements for bidirectional iterators (C++Std [bidirectional.iterators]) whose ~~value~~reference type is convertible to `mutable_buffer`.

assertion/note/pre/post-condition:

For a dereferenceable iterator, no increment, decrement, or dereference operation, or conversion of the reference type to `mutable_buffer`, shall exit via an exception.

Modify 16.2.2 [buffer.reqmts.constbuffersequence] Table 13 by adding to the third column of the first row:

expression

`net::buffer_sequence_begin(x)`
`net::buffer_sequence_end(x)`

return type

An iterator type meeting the requirements for bidirectional iterators (C++Std [bidirectional.iterators]) whose ~~value~~reference type is convertible to `const_buffer`.

assertion/note/pre/post-condition:

For a dereferenceable iterator, no increment, decrement, or dereference operation, or conversion of the reference type to `const_buffer`, shall exit via an exception.

Modify 16.7 [buffer.seq.access] to add `noexcept` to the signatures:

```
const mutable_buffer* buffer_sequence_begin(const mutable_buffer& b) noexcept;
const const_buffer* buffer_sequence_begin(const const_buffer& b) noexcept;
```

-1- *Returns:* `std::addressof(b)`.

```
const mutable_buffer* buffer_sequence_end(const mutable_buffer& b) noexcept;
const const_buffer* buffer_sequence_end(const const_buffer& b) noexcept;
```

-2- *Returns:* `std::addressof(b) + 1`.

```
template<class C> auto buffer_sequence_begin(C& c) noexcept -> decltype(c.begin());
template<class C> auto buffer_sequence_begin(const C& c) noexcept -> decltype(c.begin());
```

-3- *Returns:* `c.begin()`.

```
template<class C> auto buffer_sequence_end(C& c) noexcept -> decltype(c.end());
template<class C> auto buffer_sequence_end(const C& c) noexcept -> decltype(c.end());
```

-4- *Returns:* `c.end()`.

Add a new paragraph to 18.2.3 [socket.reqmts.native] after paragraph 1:

-2- When an operation has its effects specified as if by passing the result of `native_handle()` to a POSIX function the effect is as if `native_handle_type` is the type `int`.

Add a new paragraph to 18.7 [socket.dgram] after paragraph 5:

-6- If `native_handle_type` and `basic_socket<Protocol>::native_handle_type` are both defined then they name the same type.

Add a new paragraph to 18.8 [socket.stream] after paragraph 5:

-6- If `native_handle_type` and `basic_socket<Protocol>::native_handle_type` are both defined then they name the same type.

Modify 18.9.1 [socket.acceptor.cons] paragraph 10 as shown:

-10- *Postconditions:*

- `get_executor() == rhs.get_executor()`.
- `is_open()` returns the same value as `rhs.is_open()` prior to the constructor invocation.
- `non_blocking()` returns the same value as `rhs.non_blocking()` prior to the constructor invocation.
- `enable_connection_aborted()` returns the same value as `rhs.enable_connection_aborted()` prior to the constructor invocation.
- `native_handle()` returns the same value as `rhs.native_handle()` prior to the constructor invocation.
- `protocol_` is equal to the prior value of `rhs.protocol_`.
- `rhs.is_open() == false`.

Modify 19.1.1 [socket.streambuf.cons] as shown:

-2- *Postconditions:* `expiry() == time_point::max()` and `!error()`.

[...]

-4- *Postconditions:* `expiry() == time_point::max()` and `!error()`.

[...]

-6- *Postconditions:* Let `rhs_p` refer to the state of `rhs` just prior to this construction and let `rhs_a` refer to the state of `rhs` just after this construction.

- `is_open() == rhs_p.is_open()`
- `rhs_a.is_open() == false`
- `error() == rhs_p.error()`
- `!rhs_a.error()`
- `expiry() == rhs_p.expiry()`
- `rhs_a.expiry() == time_point::max()`

[...]

```
basic_socket_streambuf& operator=(basic_socket_streambuf&& rhs);
```

-8- *Effects:* Calls `this->close()` then moves the state from `rhs`. After the move assignment `*this` and `rhs` have ~~has~~ the observable state they ~~it~~ would have had if `*this` ~~it~~ had been move constructed from `rhs`.

Modify 21.1 [internet.synop] as shown:

```
enum class resolver_errc {
host_not_found = an implementation-defined non-zero value , // EAI_NONAME
host_not_found_try_again = an implementation-defined non-zero value , // EAI_AGAIN
service_not_found = an implementation-defined non-zero value // EAI_SERVICE };
```